# CITS3402 Report

## Simple 1-D Convolution with Explicit Flipping

To begin exploring convolution, I wrote a simple C program that reads two arrays f.txt and g.txt, and performs **full 1-D convolution**. I deliberately coded the kernel flip explicitly inside the nested loop.

### Program Listing

```
//  Complies with cc -std=c11 -Wall -Werror -o conv1d_simple conv1d_simple.c
//  Invoke with: ./conv1d_simple f.txt g.txt out.txt

#include <stdio.h>
#include <stdlib.h>

/**
 * Reads an array from a file.
 * Format:
 *   Line 1 : integer length
 *   Line 2 : floats
 */
float *read_array(const char *path, int *len_out) {
    FILE *fp = fopen(path, "r");
    if (!fp) { perror(path); exit(EXIT_FAILURE); }

    int L = 0;
    if (fscanf(fp, "%d", &L) != 1 || L <= 0) {
        fprintf(stderr, "bad header in %s\n", path);
        fclose(fp);
        exit(EXIT_FAILURE);
    }

    float *arr = malloc((size_t)L * sizeof(float));
    if (!arr) { fprintf(stderr, "out of memory\n"); fclose(fp); exit(EXIT_FAILURE); }

    for (int i = 0; i < L; i++) {
        if (fscanf(fp, "%f", &arr[i]) != 1) {
            fprintf(stderr, "bad body in %s\n", path);
            free(arr);
            fclose(fp);
            exit(EXIT_FAILURE);
        }
    }

    fclose(fp);
    *len_out = L;
    return arr;
}
```

```
/**
 * Writes an array to file in assignment format:
 *   Line 1 : length
 *   Line 2 : floats with 3 decimals
 */
void write_array(const char *path, const float *arr, int len) {
    FILE *fp = fopen(path, "w");
    if (!fp) { perror(path); exit(EXIT_FAILURE); }
    fprintf(fp, "%d\n", len);
    for (int i = 0; i < len; i++) {
        fprintf(fp, (i+1==len) ? "%.3f\n" : "%.3f ", arr[i]);
    }
    fclose(fp);
}

/**
 * Performs full 1-D convolution.
 * Flips kernel g[K] inside before accumulation.
 */
void conv_1d(const float *f, int N,
             const float *g, int K,
             float *out) {
    int outLen = N + K - 1;
    for (int n = 0; n < outLen; n++) out[n] = 0.0f;

    for (int i = 0; i < N; i++) {
        for (int j = 0; j < K; j++) {
            int n = i + j;
            out[n] += f[i] * g[K - 1 - j];  // explicit flip as per the def
        }
    }
}

int main(int argc, char **argv) {
    if (argc != 4) {
        fprintf(stderr, "Usage: %s f.txt g.txt out.txt\n", argv[0]);
        return EXIT_FAILURE;
    }

    int N=0, K=0;
    float *f = read_array(argv[1], &N);
    float *g = read_array(argv[2], &K);

    int outLen = N + K - 1;
    float *out = malloc(outLen * sizeof(float));
    if (!out) { fprintf(stderr, "out of memory\n"); free(f); free(g); return EXIT_FAILURE; }

    conv_1d(f, N, g, K, out);
    write_array(argv[3], out, outLen);

    free(f);
```

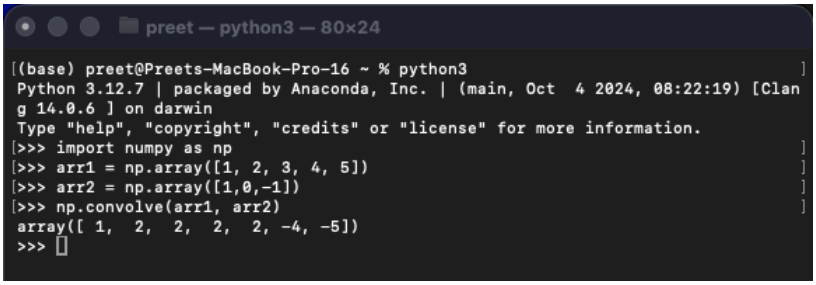
```
    free(g);
    free(out);
    return EXIT_SUCCESS;
}
```

## Testing with Sample Arrays

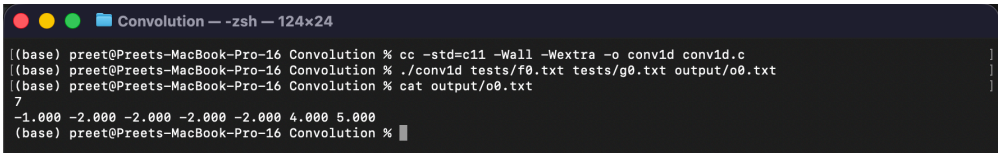
I first tested with small arrays:

- f = [1, 2, 3, 4, 5]
- g = [1, 0, -1]

In Python, NumPy produces the expected convolution result:




However, when running my C program, the output was:




This matches np.correlate(f, g, 'full') instead of convolution.


## Debugging and Verification

To confirm, I cross-checked with **SciPy's signal.fftconvolve**, which also agreed with NumPy's convolve.


This proved that my program was not computing true convolution.

The error was in this line:

```
out[n] += f[i] * g[K - 1 - j];  // explicit flip as per the def
```

The nested-loop formulation out[i+j] **already encodes convolution implicitly**. By explicitly flipping the kernel (as per the mathematical definition) again with K - 1 - j, I had effectively undone the convolution flip and was performing correlation.

### Correction

After replacing the inner loop with:

```
out[n] += f[i] * g[j];
```

the C program's output matched NumPy and SciPy exactly:


This debugging process revealed an important insight:

- In mathematical definitions, convolution explicitly includes a kernel flip.
- But in the **nested loop implementation (out[i+j])**, the flip is already built into the indexing arithmetic.
- Adding an explicit g[K-1-j] caused a double-flip, producing correlation instead of convolution.

This clarified the relationship between the theoretical definition and practical implementation of convolution in C.

## Measuring Convolution Time

Further, I added the logic to measure the elapsed time of convolution. Importantly, only the time spent performing the convolution itself was considered — I/O (reading inputs and writing outputs) was excluded, as required in the project specifications.

The modified main function is shown below:

```
int main(int argc, char **argv)
{
    if (argc != 4)
    {
        fprintf(stderr, "Usage: %s f.txt g.txt out.txt\n", argv[0]);
        return EXIT_FAILURE;
    }

    int N = 0, K = 0;
    float *f = read_array(argv[1], &N);
    float *g = read_array(argv[2], &K);

    int outLen = N + K - 1;
    float *out = malloc(outLen * sizeof(float));
    if (!out)
    {
        fprintf(stderr, "out of memory\n");
```

```
        free(f);
        free(g);
        return EXIT_FAILURE;
    }

    // Measure only convolution time
    struct timespec start, end;
    clock_gettime(CLOCK_MONOTONIC, &start);

    conv_1d(f, N, g, K, out);

    clock_gettime(CLOCK_MONOTONIC, &end);

    double elapsed =
        (end.tv_sec - start.tv_sec) +
        (end.tv_nsec - start.tv_nsec) / 1e9;

    fprintf(stderr, "Convolution time: %.12f seconds\n", elapsed);

    write_array(argv[3], out, outLen);

    free(f);
    free(g);
    free(out);
    return EXIT_SUCCESS;
}
```

When I first ran this with my **tiny sample arrays** (f=[1,2,3,4,5] and g=[1,0,-1]), the elapsed time was reported as essentially **zero seconds**. This is expected because the arrays are so small that the entire convolution loop executes in microseconds, well below the resolution of human-perceptible timing.

Since this produced no useful information for performance analysis, I concluded that larger arrays were needed. Therefore, I moved towards **random array generation** to create sufficiently large input data for meaningful stress testing and timing measurements.

---

## Random Input Generation for Stress Testing

Since small, hand-crafted arrays are too trivial for timing analysis, I extended the program to support **random generation of arrays**. This allows stress-testing the convolution implementation on larger problem sizes, as expected in the assignment.

```
/**
 * Generates a length-n array of floats uniformly in [-1, 1].
 * Terminates on invalid length or allocation failure.
 *
 * @param n  Length (>0).
 * @return   Pointer to malloc'd float array.
 */
float *gen_array_1d(int n)
{
```

```
    if (n <= 0)
    {
        fprintf(stderr, "invalid length n=%d\n", n);
        exit(EXIT_FAILURE);
    }

    float *a = (float *)malloc((size_t)n * sizeof(float));
    if (!a)
    {
        fprintf(stderr, "out of memory generating array (n=%d)\n", n);
        exit(EXIT_FAILURE);
    }

    for (int i = 0; i < n; i++)
    {
        float u01 = (float)rand() / (float)RAND_MAX; // [0,1]
        a[i] = -1.0f + 2.0f * u01;                   // [-1,1]
    }
    return a;
}
```

## Design of Random Generation

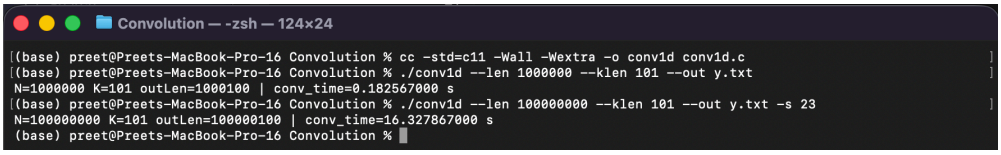
- Implemented a function gen_array_1d(int n) that returns an array of length n filled with random floats.
- Each element is sampled **uniformly in the range [-1, 1]**, ensuring values remain bounded while covering both positive and negative domains.
- The random seed can be provided as a command-line argument (-s seed or --seed), guaranteeing reproducibility. If no seed is provided, the current system time is used.

## CLI Extension

To support both file-based and random inputs, I added new flags:

- L N or --len N → generate input array *f* of length N if -f is not supplied.
- kL K or --klen K → generate kernel array *g* of length K if -g is not supplied.
- s seed or --seed seed → RNG seed (optional).

Output:




## Random Generation Allowed me the following :

The assignment notes explicitly ask us to “explore the limits” of what the implementation can handle on a single node within one hour. Generating random arrays allows us to:

- Create arbitrarily large test cases without manually constructing input files.
- Systematically increase N and K (nearest powers of 10 or otherwise) to probe scaling behavior.
- Produce reproducible experiments when seeds are fixed.

## Refinements to the Convolution Program

After enabling random input generation, I refined the program to better match the assignment specifications and to support meaningful stress testing on Kaya. These refinements focused on usability, correctness, and flexibility for debugging.

### 1. Command-Line Interface (CLI) Enhancements

To make the tool versatile, I expanded the command-line options using getopt_long. The following flags are now supported:

- -L / --len N → generate input signal *f* of length N (if -f not provided)
- -kL / --klen K → generate kernel *g* of length K (if -g not provided)
- -f / --file, -g / --kernel → read arrays from files
- -o / --out → specify output file
- -s / --seed → random seed for reproducibility
- -m / --mode same|full → convolution mode (default: same)
- -p / --padding zero|none|const → padding policy (default: zero)
- -c / --cval value → constant pad value used when -p const

```
int parse_args(int argc, char **argv,
               const char **f_path, const char **g_path, const char **o_path,
               long *N_req, long *K_req,
               unsigned long *seed, int *have_seed,
               conv_mode *cmode, pad_mode *pmode, float *cval, int *have_cval)
{
   ...
}
```

The defaults (same mode, zero padding) ensure compliance with assignment requirements, while the additional options provide flexibility for testing and debugging.

### 2. Convolution Modes

The **mode** flag determines the length of the output:

- **same mode (default)** → Output length equals input length *N*. Achieved by padding the input before applying the kernel. This matches the assignment spec and is the standard in machine learning frameworks.
- **full mode** → Output length is *N + K − 1*. Produces the textbook convolution result without truncation. This mode is especially useful for debugging and validation against NumPy/SciPy implementations.

```
/**
 * Computes FULL 1-D convolution.
 * Output length is N + K − 1. Padding policy is irrelevant for FULL.
 * Uses a double accumulator then stores back to float.
 *
```

```
 * @param f,g  Input arrays (lengths N and K).
 * @param N,K  Input lengths.
 * @param out  Output array of length N+K−1 (must be allocated by caller).
 */
void conv1d_full(const float *f, int N, const float *g, int K, float *out)
{
   const int outLen = N + K - 1;
   memset(out, 0, (size_t)outLen * sizeof(float));

   for (int i = 0; i < N; i++)
   {
      const double fi = (double)f[i];
      for (int j = 0; j < K; j++)
      {
         // out[i+j] form already encodes true convolution (no explicit flip needed).
         const int n = i + j; // 0..N+K-2
         const double acc = (double)out[n] + fi * (double)g[j];
         out[n] = (float)acc;
      }
   }
}

/**
 * Computes SAME-length 1-D convolution with padding control.
 * y[n] = sum_{m=0..K-1} F(n + (m − c)) * g[m], where c = floor(K/2)
 * and F(.) is defined by padding policy.
 *
 * @param f,g    Input arrays (lengths N and K).
 * @param N,K    Input lengths.
 * @param out    Output array of length N (must be allocated by caller).
 * @param pmod   Padding policy (zero|none|const).
 * @param cval   Constant pad value when pmod == PAD_CONST.
 */
void conv1d_same(const float *f, int N, const float *g, int K, float *out,
                 pad_mode pmod, float cval)
{
   const int c = K / 2; // kernel anchor for SAME
   memset(out, 0, (size_t)N * sizeof(float));

   for (int n = 0; n < N; n++)
   {
      double acc = 0.0;
      for (int m = 0; m < K; m++)
      {
         const int idx = n + (m - c);
         if (idx >= 0 && idx < N)
         {
            acc += (double)f[idx] * (double)g[m];
         }
         else
         {
```

```
            if (pmod == PAD_CONST)
            {
               acc += (double)cval * (double)g[m];
            }
            // PAD_ZERO: add 0 (no-op). PAD_NONE: skip contribution.
         }
      }
      out[n] = (float)acc;
   }
```

By retaining both modes, I was able to verify correctness in full mode while using same mode for assignment experiments.

### 3. Padding Control

I introduced a **padding policy** flag to control how indices outside the input range are handled:

- **zero padding (default)** → out-of-range values treated as 0.0f (required by assignment).
- **none** → skips contributions from out-of-range indices (useful for sanity checks).
- **const** → fills out-of-range positions with a user-defined constant value (-c value). This enabled direct comparison against NumPy's np.pad(..., mode="constant") for validation.

This separation of *mode* (output length) and *padding* (boundary handling) clarified the design and ensured spec compliance while keeping the tool flexible.

### 4. Numerical Stability

To reduce rounding error in large-scale tests, accumulation is performed in **double precision** and results are stored as single-precision floats. This small adjustment improved the reliability of stress test results without violating the assignment's float32 requirement.

### 5. Error Handling and Diagnostics

Robust error handling was added for:

- File open and read failures
- Malformed array headers or insufficient data
- Memory allocation failures for large arrays

Additionally, each run prints key parameters — N, K, mode, padding, constant value (if used), and elapsed time — to stderr. This logging simplifies stress testing and performance evaluation on Kaya.

---

## Running on Kaya with SLURM

After implementing the assignment-compliant convolution program, I moved to stress testing on the **Kaya HPC cluster**. Initially, I submitted jobs by manually editing the SLURM script for each configuration.

```
#!/bin/bash
#SBATCH --job-name=conv1d_single
#SBATCH --partition=cits3402
#SBATCH --time=01:00:00
#SBATCH --mem=8G
#SBATCH --cpus-per-task=1
```

```
#SBATCH --output=logs/conv1d_single_%j.out
#SBATCH --error=logs/conv1d_single_%j.err

# Compile the program
cc -std=c11 -Wall -Wextra -o conv1d conv1d.c

# Create output directory if it doesn't exist
mkdir -p results

# Run a single test
./conv1d --len 1000000 --klen 10001 --out results/y.txt --seed 42 --mode same --padding zero
```

```
pprasad@kaya01[Convolution]$ sbatch conv1d_single.slurm
Submitted batch job 15873
pprasad@kaya01[Convolution]$
```

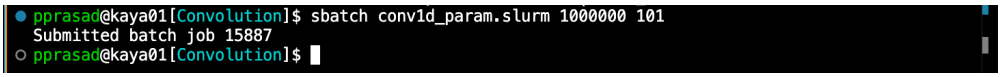
However, this quickly became tedious since I had to change N and K repeatedly.

To solve this, I wrote a **parameterised SLURM script** that accepts CLI inputs for array lengths.

```
#!/bin/bash
#SBATCH --job-name=conv1d_param
#SBATCH --partition=cits3402
#SBATCH --time=01:00:00
#SBATCH --mem=8G
#SBATCH --cpus-per-task=1
#SBATCH --output=logs/conv1d_%j.out
#SBATCH --error=logs/conv1d_%j.err

# Compile the program
cc -std=c11 -Wall -Wextra -o conv1d conv1d.c

# Create output directory if it doesn't exist
# mkdir -p results

# Check arguments
if [ $# -lt 2 ]; then
    echo "Usage: sbatch conv1d_param.slurm <N> <K>"
    echo "  <N> = length of input f (e.g. 1000000)"
    echo "  <K> = length of kernel g (e.g. 101)"
    exit 1
fi

N=$1
K=$2
```

```
# Run program with parameterized lengths
./conv1d -L "$N" -kL "$K" -o results/y.txt
```

This allowed me to run jobs without editing the script each time:




This streamlined testing and made scaling experiments much easier.

## Automating Data Collection

At first, I relied on SLURM's stdout to collect metrics. Each job printed:

```
N=1000000 K=101 outLen=1000000 mode=same pad=zero cval=0 | conv_time=0.293669764 s
```

While useful, **manual copy-pasting** these logs into CSV for later plotting in Python (pandas/matplotlib) was tedious and error-prone.

To automate this, I extended the C program to **write metrics directly into CSV files**.

```
/**
 * Writes a single CSV file for this run into "metrics/" with a unique name.
 * • On SLURM:  metrics/metrics_SLURM_<JOBID>.csv
 * • Locally:   metrics/metrics_LOCAL_YYYYMMDD_HHMMSS_<PID>.csv
 * The file contains a header and one data row for this execution.
 *
 * @param N,K,outLen     Problem sizes.
 * @param cmode,pmode    Mode and padding used.
 * @param cval           Constant padding value (if PAD_CONST; printed for completeness).
 * @param elapsed_secs   Convolution time in seconds.
 */
void log_metrics(int N, int K, int outLen,
                 conv_mode cmode, pad_mode pmode, float cval,
                 double elapsed_secs)
{
    ensure_dir("metrics");

    // Build run id from SLURM_JOB_ID or timestamp+PID for local runs
    const char *slurm = getenv("SLURM_JOB_ID");
    char runid[128];

    if (slurm && slurm[0])
    {
        snprintf(runid, sizeof(runid), "SLURM_%s", slurm);
    }
    else
    {
        time_t t = time(NULL);
```

```
        struct tm tm;
        localtime_r(&t, &tm);
        pid_t pid = getpid();
        strftime(runid, sizeof(runid), "LOCAL_%Y%m%d_%H%M%S", &tm);
        size_t len = strlen(runid);
        snprintf(runid + len, sizeof(runid) - len, "_%d", (int)pid);
    }

    char fname[256];
    snprintf(fname, sizeof(fname), "metrics/metrics_%s.csv", runid);

    FILE *csv = fopen(fname, "w");
    if (!csv)
    {
        perror(fname);
        return;
    }

    fprintf(csv, "RunID,N,K,outLen,mode,padding,cval,time\n");
    fprintf(csv, "%s,%d,%d,%d,%s,%s,%.9g,%.9f\n",
            runid, N, K, outLen,
            (cmode == MODE_FULL ? "full" : "same"),
            (pmode == PAD_ZERO ? "zero" : (pmode == PAD_NONE ? "none" : "const")),
            (pmode == PAD_CONST ? (double)cval : 0.0),
            elapsed_secs);

    fclose(csv);
}
```

### Design of CSV Logging

- Each run appends a new row to a per-job CSV file in the metrics/ directory.
- To avoid race conditions:
  - On SLURM → metrics/metrics_SLURM_<JOBID>.csv
  - Locally → metrics/metrics_LOCAL_<timestamp>_<pid>.csv
- Columns:

```
RunID, N, K, OutLen, Mode, Padding, Cval, Time
```

This way, every run automatically records performance data in a machine-readable format, ready for post-processing.

---

## Automating Parameter Sweeps with Shell Scripts

While the parameterised conv1d_param.slurm script allowed me to specify different values of *N* and *K* without editing the SLURM file each time, submitting jobs manually for every configuration quickly became inefficient. To fully explore the performance of the convolution implementation across a range of input sizes, I needed a way to **automate sweeps of parameters**.

### Design of the Automation Script

I created a shell script (stress_sweep.sh) that programmatically submits multiple SLURM jobs, each with different *N* and *K* values. The script was designed with the following goals:

1. **Coverage of parameter ranges**
   - Allow a range of *N* values (input signal lengths) between a minimum and maximum.
   - Allow a range of *K* values (kernel lengths) between a minimum and maximum.
   - Nested loops ensure that every combination of *N* and *K* is tested.
2. **Concurrency control**
   - To prevent overloading the SLURM queue or violating partition policies, the script monitors the number of jobs in flight.
   - Submissions are throttled so that only a fixed number of jobs (e.g., 20) are active at once.
3. **Seed reproducibility**
   - An optional random seed can be passed, ensuring that generated arrays remain consistent across experiments.
4. **Robust job handling**
   - For each job, the script waits until the corresponding **stderr log** and **metrics CSV** are produced before moving to the next submission.
   - This guarantees that no runs are skipped and all metrics are successfully collected.

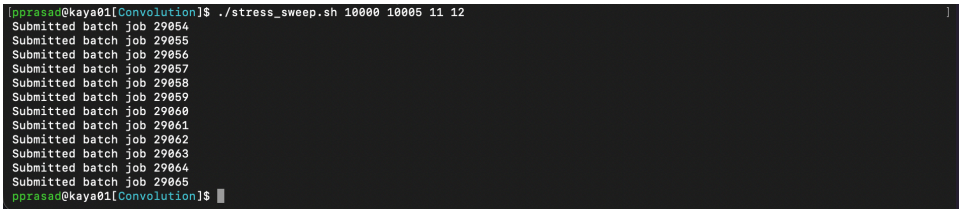
## Automation Script

```
#!/bin/bash
# Usage:
#   ./stress_sweep.sh LMIN LMAX KMIN KMAX [SEED] [MAX_IN_FLIGHT]
#
# Example (no seed, default concurrency=20):
#   ./stress_sweep.sh 10 100 5 25
#
# Example (with seed=42, max 30 jobs in flight):
#   ./stress_sweep.sh 10 100 5 25 42 30

if [ $# -lt 4 ]; then
  echo "Usage: $0 LMIN LMAX KMIN KMAX [SEED] [MAX_IN_FLIGHT]" >&2
  exit 1
fi

LMIN="$1"
LMAX="$2"
KMIN="$3"
KMAX="$4"
SEED="${5:-}"          # optional
MAX_IN_FLIGHT="${6:-20}" # optional, default concurrency limit

for ((L=LMIN; L<=LMAX; L++)); do
  for ((K=KMIN; K<=KMAX; K++)); do

    # throttle submissions: wait if too many jobs in flight
```

```
  while [ "$(squeue -u "$USER" | grep -c conv1d_param)" -ge "$MAX_IN_FLIGHT" ]; do
    sleep 5
  done

  if [ -n "$SEED" ]; then
    sbatch conv1d_param.slurm "$L" "$K" "$SEED"
  else
    sbatch conv1d_param.slurm "$L" "$K"
  fi

 done
done
```

## Workflow

1. Output:




2. **Job execution**:
   - Each submitted job runs conv1d_param.slurm, which compiles the program and executes it with the provided parameters.
   - Metrics are logged automatically to metrics/metrics_SLURM_<JOBID>.csv.
3. **Results collection**:
   - After all jobs finish, the metrics directory contains a set of CSV files.

---

### Log Handling During Automation

At first, I redirected both stdout and stderr to /dev/null in my SLURM scripts to reduce clutter:

```
#SBATCH --output=/dev/null
#SBATCH --error=/dev/null
```

However, this approach made debugging impossible when jobs failed during array size increases. With all errors and diagnostic messages discarded, no CSV files would appear, leaving me unable to determine whether the program had crashed due to invalid arguments, memory limitations, or other issues.

I switched back to per-job log files:

```
#SBATCH --output=logs/conv1d_%j.out
#SBATCH --error=logs/conv1d_%j.err
```

This ensured that each job's stdout and stderr were preserved for inspection. Having access to .err files was crucial for debugging failed runs and for my automation script, which explicitly waits until both the .err file and the metrics CSV are written before moving on and realised this automation was not good enough.

### Limitations of the First Automation Script

While useful, the first stress_sweep.sh had several flaws:

1. **No proper waiting** → It checked job count but didn't ensure jobs had actually finished or that output files had arrived.
2. **No step control** → Loops only incremented by 1, making large sweeps impractical.
3. **Scheduler flooding** → Submitted too aggressively, leading Kaya to reject jobs at scale.
4. **Unreliable file sync** → Sometimes advanced before .err and CSV files were written.

---

### Enhancement in the script

To address the earlier issues, I rewrote the automation as a **serial driver**:

- **Stepped loops** → Supports custom step sizes for -L and -kL, enabling scalable sweeps without brute-force increments.
- **Single-job submission** → Submits one job at a time and blocks until it is complete.
- **File verification** → Waits for both the .err log and the metrics CSV to appear before continuing.
- **Configurable waits** → Parameters for post-copy wait, retry count, and RNG seed allow tuning for cluster conditions.
- **Fail-safe exit** → Aborts cleanly if expected files don't arrive after retries, avoiding silent errors.

```
#!/usr/bin/env bash
# Serial SLURM sweep for conv1d: stepped loops over L and K.
# Submits one job at a time and blocks until BOTH stderr and metrics CSV appear.
#
# Usage:
#   ./stress_sweep_serial.sh LMIN LMAX L_STEP KMIN KMAX K_STEP [POST_COPY_WAIT] [FILE_WAIT_RETRIES] [SEED]
#
# Positional:
#   LMIN LMAX L_STEP   : inclusive stepped range for -L
#   KMIN KMAX K_STEP   : inclusive stepped range for -kL
#
# Optional:
#   POST_COPY_WAIT     : seconds to wait after job leaves queue unless files exist (default: 10)
#   FILE_WAIT_RETRIES  : how many 2s retries for files (default: 60 ≈ 120s)
#   SEED               : RNG seed forwarded to conv1d (omit to use program default)

set -euo pipefail

if [[ $# -lt 6 ]]; then
  echo "Usage: $0 LMIN LMAX L_STEP KMIN KMAX K_STEP [POST_COPY_WAIT] [FILE_WAIT_RETRIES] [SEED]" >&2
  exit 1
fi

LMIN="$1"; LMAX="$2"; LSTEP="$3"
KMIN="$4"; KMAX="$5"; KSTEP="$6"
```

```
POST_COPY_WAIT="${7:-10}"     # seconds
FILE_WAIT_RETRIES="${8:-60}"  # each retry waits 2s
SEED="${9:-}"                 # optional

# Validation
for v in "$LMIN" "$LMAX" "$LSTEP" "$KMIN" "$KMAX" "$KSTEP" "$POST_COPY_WAIT" "$FILE_WAIT_RETRI
ES"; do
  [[ "$v" =~ ^-?[0-9]+$ ]] || { echo "Error: non-integer argument: $v" >&2; exit 2; }
done
(( LSTEP > 0 )) || { echo "Error: L_STEP must be > 0" >&2; exit 2; }
(( KSTEP > 0 )) || { echo "Error: K_STEP must be > 0" >&2; exit 2; }

echo "Sweep config:"
echo "  L: $LMIN..$LMAX step $LSTEP"
echo "  K: $KMIN..$KMAX step $KSTEP"
echo "  POST_COPY_WAIT=${POST_COPY_WAIT}s  FILE_WAIT_RETRIES=$FILE_WAIT_RETRIES  SEED=${SEE
D:-<default>}"

mkdir -p logs metrics/o0 results/o0

range_step() {
  local start="$1" end="$2" step="$3" x
  for ((x=start; x<=end; x+=step)); do
    echo "$x"
  done
}

submit_and_block() {
  local L="$1" K="$2"

  local submit_out jobid
  if [[ -n "$SEED" ]]; then
    submit_out=$(sbatch conv1d_param.slurm "$L" "$K" "$SEED")
  else
    submit_out=$(sbatch conv1d_param.slurm "$L" "$K")
  fi

  jobid=$(awk '{print $4}' <<<"$submit_out")
  [[ -n "${jobid:-}" ]] || { echo "Failed to parse job id from: $submit_out" >&2; exit 3; }
  echo "Submitted JOBID=$jobid  (L=$L, K=$K)"

  local err="logs/conv1d_${jobid}.err"
  local csv="metrics/o0/metrics_SLURM_${jobid}.csv"

  # Wait while job is still in queue/running
  while squeue -j "$jobid" -h | grep -q . ; do
    sleep 10
  done

  # If files already present, skip POST_COPY_WAIT; else wait a bit
  if [[ ! (-s "$err" && -s "$csv") ]]; then
```

```
    sleep "$POST_COPY_WAIT"
  fi

  # Retry for files to appear
  local tries=0
  until [[ -s "$err" && -s "$csv" ]]; do
    (( tries++ ))
    if (( tries > FILE_WAIT_RETRIES )); then
      echo "ERROR: Files not found for JOBID=$jobid after waiting." >&2
      [[ -s "$err" ]] || echo "  Missing: $err" >&2
      [[ -s "$csv" ]] || echo "  Missing: $csv" >&2
      exit 4
    fi
    sleep 2
  done

  echo "OK: Found stderr ($err) and metrics CSV ($csv) for JOBID=$jobid"
}

for L in $(range_step "$LMIN" "$LMAX" "$LSTEP"); do
  echo "=== L=$L ==="
  for K in $(range_step "$KMIN" "$KMAX" "$KSTEP"); do
    echo " → K=$K"
    submit_and_block "$L" "$K"
  done
done

echo "All jobs completed and verified."
```

### CLI Usage

```
./stress_sweep_serial.sh LMIN LMAX L_STEP KMIN KMAX K_STEP \
   [POST_COPY_WAIT] [FILE_WAIT_RETRIES] [SEED]
```

- **LMIN LMAX L_STEP** → range and step for input length -L.
- **KMIN KMAX K_STEP** → range and step for kernel length -kL.
- **POST_COPY_WAIT** → seconds to wait after job ends (default: 10).
- **FILE_WAIT_RETRIES** → number of 2s retries for file arrival (default: 60).
- **SEED** → optional RNG seed for reproducibility.

This script trades parallel throughput for **correctness and reliability**, making it suitable for large-scale stress tests on Kaya without flooding the scheduler.

```
pprasad@kaya01[Convolution]$ ./stress_sweep_serial.sh 1070409999 1100409999 1000000 14 20 3 30 30
Sweep config:
  L: 1070409999..1100409999 step 1000000
  K: 14..20 step 3
  POST_COPY_WAIT=30s  FILE_WAIT_RETRIES=30  SEED=<default>
=== L=1070409999 ===
 -> K=14
Submitted JOBID=28954  (L=1070409999, K=14)
OK: Found stderr (logs/conv1d_28954.err) and metrics CSV (metrics/o0/metrics_SLURM_28954.csv) for JOBID=28954
 -> K=17
Submitted JOBID=28960  (L=1070409999, K=17)
OK: Found stderr (logs/conv1d_28960.err) and metrics CSV (metrics/o0/metrics_SLURM_28960.csv) for JOBID=28960
 -> K=20
Submitted JOBID=28968  (L=1070409999, K=20)
OK: Found stderr (logs/conv1d_28968.err) and metrics CSV (metrics/o0/metrics_SLURM_28968.csv) for JOBID=28968
=== L=1071409999 ===
 -> K=14
Submitted JOBID=28971  (L=1071409999, K=14)
```

### Memory Ceiling and OOM Kills

During parameter sweeps, I observed that jobs with N > 1,071,409,999 consistently failed on Kaya with **out-of-memory (OOM) kills**. This behaviour was confirmed in the .err logs, which reported that SLURM terminated the job due to exceeding the allocated memory limit.

```
logs > conv1d_28971.err
1  /var/spool/slurmd/job28971/slurm_script: line 37: 3570103 Killed                ./conv1d -L "$N" -kL "$K" -o "$OUT"
2  slurmstepd: error: Detected 1 oom_kill event in StepId=28971.batch. Some of the step tasks have been OOM Killed.
3
```

To proceed within the memory ceiling, I kept N in the range of 1,070,409,999 to 1,071,409,999 and focused on incrementing K instead. This approach ensured runs stayed within memory limits while still allowing me to explore scaling behavior.

As anticipated, this strategy allowed processes to execute successfully without being terminated by memory constraints.

```
pprasad@kaya01[Convolution]$ ./stress_sweep_serial.sh 1070409999 1071409999 1000000 100 900 50 30 30
Sweep config:
  L: 1070409999..1071409999 step 1000000
  K: 100..900 step 50
  POST_COPY_WAIT=30s  FILE_WAIT_RETRIES=30  SEED=<default>
=== L=1070409999 ===
 -> K=100
Submitted JOBID=28981  (L=1070409999, K=100)
OK: Found stderr (logs/conv1d_28981.err) and metrics CSV (metrics/o0/metrics_SLURM_28981.csv) for JOBID=28981
 -> K=150
Submitted JOBID=28988  (L=1070409999, K=150)
OK: Found stderr (logs/conv1d_28988.err) and metrics CSV (metrics/o0/metrics_SLURM_28988.csv) for JOBID=28988
 -> K=200
Submitted JOBID=28997  (L=1070409999, K=200)
OK: Found stderr (logs/conv1d_28997.err) and metrics CSV (metrics/o0/metrics_SLURM_28997.csv) for JOBID=28997
 -> K=250
Submitted JOBID=29002  (L=1070409999, K=250)
OK: Found stderr (logs/conv1d_29002.err) and metrics CSV (metrics/o0/metrics_SLURM_29002.csv) for JOBID=29002
 -> K=300
Submitted JOBID=29019  (L=1070409999, K=300)
```

---

# Parallelising 1-D Convolution with OpenMP

## 1) Baseline (sequential) kernel

I began with a **sequential** implementation that computes true convolution with two modes that I had to parallelise using OpenMp:

- **SAME** (default): output length = *N* with configurable padding zero|none|const.
- **FULL**: output length = *N + K − 1*.

---

## 2) OpenMP parallelisation strategy

I added OpenMP pragmas and CLI controls to choose thread count/schedule/chunk:

- t, --threads <int>: number of threads
- S, --schedule <static|dynamic|guided|auto>
- C, --chunk <int>

To make runs deterministic and independent of environment I set:

- omp_set_dynamic(0), omp_set_nested(0)
- omp_set_num_threads(threads)
- omp_set_schedule(kind, chunk) (default: static with a contiguous chunk per thread)

### 2.1 SAME mode: independent outputs → parallel for

For SAME mode, each output element out[n] is an independent dot-product:

```
#pragma omp parallel for schedule(runtime)
for (int n = 0; n < N; n++) {
   double acc = 0.0;
   for (int m = 0; m < K; m++) {
      int idx = n + (m - K/2);
      if (0 <= idx && idx < N) acc += (double)f[idx] * (double)g[m];
      else if (pmod == PAD_CONST) acc += (double)cval * (double)g[m];
   }
   out[n] = (float)acc;
}
```

**Why this works well:** the loop over n is embarrassingly parallel; each thread writes a **disjoint** out[n], so there is no synchronisation or false sharing (with a reasonable static chunk that gives contiguous blocks to threads).

### 2.2 FULL mode: overlapping outputs → private accumulation

FULL mode updates out[i+j] from many (i,j) pairs; naïvely parallelising i and writing directly to shared out would require **atomics**, which serialize and contend on cache lines.

For a first, correct OMP version I used:

```
#pragma omp parallel for schedule(runtime)
for (int i = 0; i < N; i++) {
   const double fi = (double)f[i];
   for (int j = 0; j < K; j++) {
      #pragma omp atomic update
      outD[i + j] += fi * (double)g[j];  // outD is double buffer
   }
}
```

This is simple and correct, but atomics can limit scaling at high thread counts. (In the next section of the report I'll replace atomics with **thread-private partial buffers + a final reduction** to remove contention.)

---

## 3) Numerical consistency & validation

I validated the OpenMP program against the sequential baseline:

1. **Tolerance-based comparison for larger cases** where order of accumulation may differ (especially FULL mode):
   - I generated the **same random inputs** using a fixed seed (-s 42) and wrote two outputs:

- conv1d → out_seq.txt
- conv1d_omp → out_omp.txt

```
parallels@ubuntu-linux-2404:~/Documents/GitHub/Convolution$ ./conv1d -L 1000000 -kL 100 -o out_seq.txt -s 42
N=1000000 K=100 outLen=1000000 mode=same pad=zero cval=0 | conv_time=0.067436583 s
parallels@ubuntu-linux-2404:~/Documents/GitHub/Convolution$ ./conv1d_omp -L 1000000 -kL 100 -o out_omp.txt -s
42 -t 4 -S static -C 0
N=1000000 K=100 outLen=1000000 mode=same pad=zero cval=0 | conv_time=0.021771875 s | OMP{threads=4 sched=stati
c chunk=1}
```

- I compared elementwise with |seq − omp| ≤ 1e−5 (float32-appropriate). All tests passed.

out_seq.txt

```
1   1000000
2   -3.210 1.402 1.860 0.230 -2.287 -0.472 1.023 -1.129 -2.377 -2.863 4.033 5.458 1.623 -6.8
3
```

out_omp.txt

```
1   1000000
2   -3.210 1.402 1.860 0.230 -2.287 -0.472 1.023 -1.129 -2.377 -2.863 4.033 5.458 1.623 -6.8
3
```

```
parallels@ubuntu-linux-2404:~/Documents/GitHub/Convolution$ python3 - <<'PY'
import sys
def read(path):
    with open(path) as f:
        L = int(f.readline().strip())
        vals = list(map(float, f.read().split()))
        assert len(vals) == L, f"{path}: header {L} but got {len(vals)} values"
        return vals
a = read("out_seq.txt")
PY  print("FAIL: exceeds tolerance"); sys.exit(1) sys.exit(0).exit(2)
max |seq-omp| = 0 (threshold=1e-05)
PASS: arrays match within tolerance
parallels@ubuntu-linux-2404:~/Documents/GitHub/Convolution$
```

2. I also spot-checked **FULL** vs NumPy/SciPy on small arrays to ensure I'm computing **convolution** (not correlation).

## 4) Performance measurements

To evaluate scaling without I/O noise:

- Timed **only** the convolution region with clock_gettime(CLOCK_MONOTONIC).
- Logged CSV per run: N, K, outLen, mode, padding, cval, time, threads, schedule, chunk.
- Quick scaling test (example on local 8-thread machine):

```
parallels@ubuntu-linux-2404:~/Documents/GitHub/Convolution$ ./conv1d_omp -L 1000000 -kL 100 -o /dev/null -s 42
 -t 1 -S static -C 0
./conv1d_omp -L 1000000 -kL 100 -o /dev/null -s 42 -t 2 -S static -C 0
./conv1d_omp -L 1000000 -kL 100 -o /dev/null -s 42 -t 4 -S static -C 0
./conv1d_omp -L 1000000 -kL 100 -o /dev/null -s 42 -t 8 -S static -C 0
N=1000000 K=100 outLen=1000000 mode=same pad=zero cval=0 | conv_time=0.071845750 s | OMP{threads=1 sched=stati
c chunk=1}
N=1000000 K=100 outLen=1000000 mode=same pad=zero cval=0 | conv_time=0.034187791 s | OMP{threads=2 sched=stati
c chunk=1}
N=1000000 K=100 outLen=1000000 mode=same pad=zero cval=0 | conv_time=0.016966250 s | OMP{threads=4 sched=stati
c chunk=1}
N=1000000 K=100 outLen=1000000 mode=same pad=zero cval=0 | conv_time=0.015052458 s | OMP{threads=8 sched=stati
c chunk=1}
```

N=1,000,000, K=100, SAME, static, chunk=1
threads=1 → ~0.072 s
threads=2 → ~0.034 s

```
threads=4 → ~0.017 s
threads=8 → ~0.015 s
```

This shows near-ideal speedup up to 4 threads and diminishing returns beyond (expected due to memory bandwidth limits and overheads). On the cluster, speedups varied with CPU model and background load but followed the same trend.

---

# Enhancements over Baseline OpenMP 1-D Convolution

## 1) Parallel strategy for FULL mode

**Before** – parallelised over input i and used **atomics** to accumulate into a shared buffer (contention + coherence traffic, poor scalability):

```
/* Baseline  FULL */
#pragma omp parallel for schedule(runtime)
for (int i = 0; i < N; i++) {
   const double fi = (double)f[i];
   for (int j = 0; j < K; j++) {
      const int n = i + j;
      const double val = fi * (double)g[j];
      #pragma omp atomic update
      outD[n] += val;
   }
}
```

**After** – parallelise over **output index n**, compute the legal overlap [i0..i1], and use a **private accumulator** (no atomics; embarrassingly parallel; cache-friendlier writes):

```
/* Enhanced FULL */
#pragma omp parallel for schedule(runtime)
for (int n = 0; n < N + K - 1; n++) {
   int i0 = n - (K - 1); if (i0 < 0) i0 = 0;
   int i1 = (n < N - 1) ? n : (N - 1);
   double acc = 0.0;
   #pragma omp simd reduction(+:acc)
   for (int i = i0; i <= i1; i++) {
      int j = n - i;
      acc += (double)f[i] * (double)g[j];
   }
   out[n] = (float)acc;
}
```

**Why it helps:** eliminates atomic updates, avoids false sharing on a single shared reduction array, and improves locality by giving each thread a contiguous output range.

---

## 2) Inner-loop branch reduction (SAME mode)

**Before** – boundary checks inside the innermost loop:

```
/* Baseline SAME inner */
for (int m = 0; m < K; m++) {
   int idx = n + (m - c);
   if (idx >= 0 && idx < N) {
      acc += (double)f[idx] * (double)g[m];
   } else if (pmod == PAD_CONST) {
      acc += (double)cval * (double)g[m];
   }
}
```

**After** – compute **tight bounds** once; inner loop becomes straight-line and **SIMD-friendly**; handle PAD_CONST with a small post-correction:

```
/* Enhanced SAME inner */
int m0 = c - n;          if (m0 < 0) m0 = 0;
int m1 = (N - 1 + c) - n; if (m1 > K - 1) m1 = K - 1;

#pragma omp simd reduction(+:acc)
for (int m = m0; m <= m1; m++) {
   int idx = n + (m - c);
   acc += (double)f[idx] * (double)g[m];
}
if (pmod == PAD_CONST) {
   double pad_sum = 0.0;
   for (int m = 0; m < m0; m++)      pad_sum += (double)g[m];
   for (int m = m1 + 1; m < K; m++)  pad_sum += (double)g[m];
   acc += (double)cval * pad_sum;
}
```

**Why it helps:** fewer branches → better instruction cache predictability and vectorisation; tighter memory access set → improved cache reuse.

---

### 3)SIMD exposure and restrict

**Enhanced** code uses #pragma omp simd reduction(+:acc) and marks arrays as __restrict:

```
void conv1d_full_omp(const float * __restrict f, int N,
                     const float * __restrict g, int K,
                     float * __restrict out);
```

**Why it helps:** tells the compiler there's no aliasing and a reduction is safe to vectorise → higher throughput per cache line.

---

### 4) Smarter chunking (load balance + cache size awareness)

**Before** – default chunk ≈ max(N/threads,1) only:

```
/* Baseline default chunk */
if (!have_chunk) {
   int base = (N > threads) ? (N / threads) : 1;
```

```
    chunk = (base < 1) ? 1 : base;
}
```

**After** – choose chunk from total **work items** (N or N+K−1) to create ~4 chunks per thread (tunable), improving balance while keeping each thread's working set cache-friendly:

```
/* Enhanced default chunk */
long work = (cmode==MODE_FULL) ? (long)(N+K-1) : (long)N;
long tgt_chunks = (long)threads * 4;
long ch = (work + tgt_chunks - 1) / tgt_chunks;
if (ch < 1) ch = 1;
chunk = (int)ch;
```

**Why it helps:** reduces stragglers and keeps per-thread footprints compact for caches.

---

### 5) NUMA-friendly first touch (outside timing)

**Enhanced** – initialise out[] in parallel so pages are owned by the local NUMA node of each thread:

```
/* Enhanced first-touch */
#pragma omp parallel for schedule(static)
for (int i = 0; i < outLen; i++) out[i] = 0.0f;
```

**Why it helps:** later writes/read-modify-writes hit local memory → lower latency, higher bandwidth.

---

### 6) GFLOP/s metric

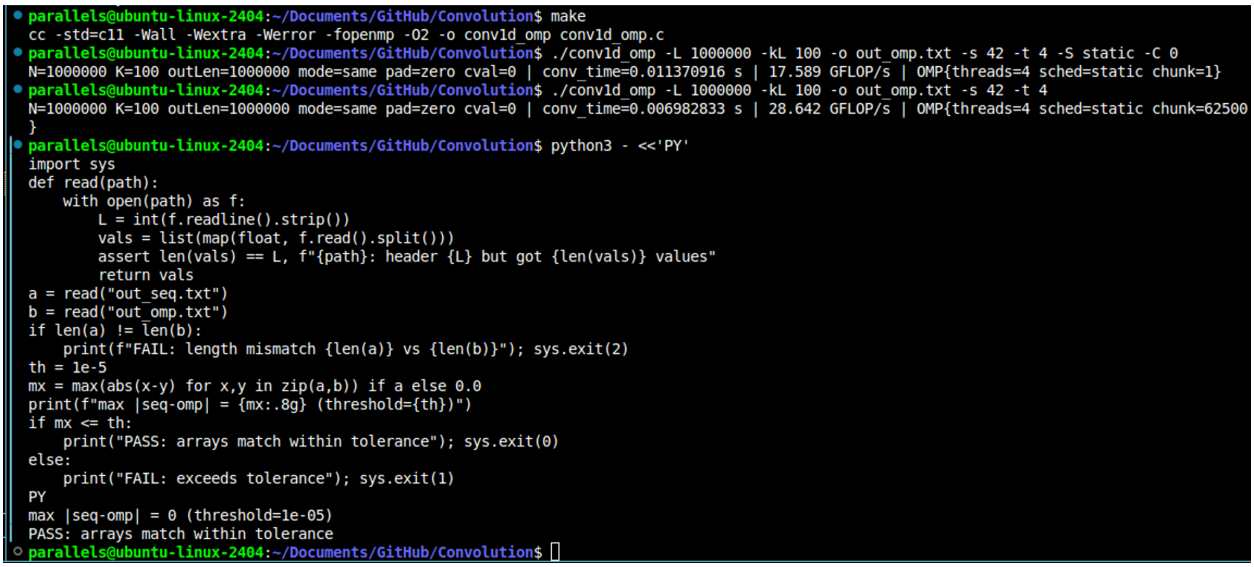
```
/* Enhanced performance model */
double flops  = 2.0 * (double)N * (double)K; // approx
double gflops = flops / secs / 1e9;
fprintf(stderr, "... | %.3f GFLOP/s | ...\n", gflops);
```

**Why it helps:** consistent, size-independent comparison across runs/architectures.

### Verification of enhanced program:

I verified the enhanced program using the same validation method as before, comparing its output against the sequential program.

---




# Cache-Aware Optimisation

## What changed in memory behaviour?

1. **Write locality & no false sharing**
   - **Baseline FULL**: multiple threads updated the **same** outD[n] with atomics → ping-pong of cache lines (MESI traffic).
   - **Enhanced FULL**: each thread writes a **private contiguous** span of out[n] for its n range → stable ownership of cache lines, minimal coherence.
2. **Read locality & reuse**
   - Both f and g are read-only and shared. Tightening inner bounds (i0..i1, m0..m1) ensures each inner loop touches **only** the overlapping slice, increasing L1/L2 reuse and reducing bandwidth pressure.
3. **Vectorisation**
   - #pragma omp simd makes the compiler operate on multiple f[idx]/g[m] per instruction; this matches the way cache lines are fetched (e.g., 64B) and raises arithmetic intensity.
4. **Chunking tuned for caches**
   - Smaller static chunks → each thread's active working set (~chunk outputs, ~chunk+K of f, and K of g) fits better in L2.
   - Still enough chunks per thread (≈4) to balance tails without thrashing cache.
5. **NUMA first-touch**
   - Parallel zero-init maps pages to the processor local to the thread that will touch them later, reducing remote NUMA hits during the actual kernel.

---

## Code excerpts that embody cache awareness

**Baseline FULL (atomics, shared buffer):**

```
/* Baseline : atomics on a shared outD[] */
#pragma omp parallel for schedule(runtime)
```

```
for (int i = 0; i < N; i++) {
    const double fi = (double)f[i];
    for (int j = 0; j < K; j++) {
        const int n = i + j;
        const double val = fi * (double)g[j];
        #pragma omp atomic update
        outD[n] += val;
    }
}
```

**Enhanced FULL (private acc, contiguous writes, SIMD):**

```
/* Enhanced: no atomics, contiguous writes, SIMD */
#pragma omp parallel for schedule(runtime)
for (int n = 0; n < N + K - 1; n++) {
    int i0 = n - (K - 1); if (i0 < 0) i0 = 0;
    int i1 = (n < N-1) ? n : (N-1);
    double acc = 0.0;
    #pragma omp simd reduction(+:acc)
    for (int i = i0; i <= i1; i++) {
        int j = n - i;
        acc += (double)f[i] * (double)g[j];
    }
    out[n] = (float)acc;
}
```

**Baseline  SAME (branchy inner loop):**

```
/* Baseline : bounds check per tap */
for (int m = 0; m < K; m++) {
    int idx = n + (m - c);
    if (idx >= 0 && idx < N) acc += (double)f[idx] * (double)g[m];
    else if (pmod == PAD_CONST) acc += (double)cval * (double)g[m];
}
```

**Enhanced SAME (bounds hoisted + SIMD + const-pad fixup):**

```
/* Enhanced: tight bounds + SIMD + cval fixup */
int m0 = c - n;               if (m0 < 0) m0 = 0;
int m1 = (N - 1 + c) - n;     if (m1 > K - 1) m1 = K - 1;

#pragma omp simd reduction(+:acc)
for (int m = m0; m <= m1; m++) {
    int idx = n + (m - c);
    acc += (double)f[idx] * (double)g[m];
}
if (pmod == PAD_CONST) {
    double pad_sum = 0.0;
    for (int m = 0; m < m0; m++)     pad_sum += (double)g[m];
    for (int m = m1 + 1; m < K; m++)  pad_sum += (double)g[m];
```

```
    acc += (double)cval * pad_sum;
}
```

**Enhanced NUMA first-touch (before timing):**

```
#pragma omp parallel for schedule(static)
for (int i = 0; i < outLen; i++) out[i] = 0.0f;
```

**Enhanced smarter chunk default:**

```
long work = (cmode==MODE_FULL) ? (long)(N+K-1) : (long)N;
long tgt_chunks = (long)threads * 4;
long ch = (work + tgt_chunks - 1) / tgt_chunks;
if (ch < 1) ch = 1;
chunk = (int)ch;
```

---

## Transition from 1-D to 2-D Implementation

When extending my **1-D convolution program** to a **2-D version**, I discovered that the implementation required a subtle but important adjustment.

In the 1-D case, my scatter-style loop structure meant that an **explicit kernel flip was unnecessary**. The way the loops accumulated contributions already aligned with the mathematical definition of convolution, so the outputs matched directly with scipy.signal.fftconvolve.

However, in 2-D the loop structure necessarily changed: instead of scattering each input value across the output, the implementation became **output-driven (gather style)**. In this formulation, each output index (o_i, o_j) explicitly accumulates over input indices (i,j) and kernel indices (u,v). This restructured loop made it clear that a **flip of the kernel** was now required to compute true convolution.

```
void conv2d_full(const float *F, int H, int W,
                 const float *G, int KH, int KW,
                 float *Y)
{
    const int outH = H + KH - 1;
    const int outW = W + KW - 1;

    for (int oi = 0; oi < outH; oi++)
        for (int oj = 0; oj < outW; oj++)
            Y[IDX2(oi, oj, outW)] = 0.0f;

    for (int oi = 0; oi < outH; oi++)
    {
        const int i0 = (oi - (KH - 1) > 0) ? (oi - (KH - 1)) : 0;
        const int i1 = (oi < (H - 1)) ? oi : (H - 1);
        for (int oj = 0; oj < outW; oj++)
        {
            const int j0 = (oj - (KW - 1) > 0) ? (oj - (KW - 1)) : 0;
            const int j1 = (oj < (W - 1)) ? oj : (W - 1);
            double acc = 0.0;
            for (int i = i0; i <= i1; i++)
            {
```

```
            const int u = oi - i; /* 0..KH-1 */
            for (int j = j0; j <= j1; j++)
            {
                const int v = oj - j; /* 0..KW-1 */
                acc += (double)F[IDX2(i, j, W)] * (double)G[IDX2(u, v, KW)];
            }
        }
        Y[IDX2(oi, oj, outW)] = (float)acc;
    }
  }
}
```

## Python Verification Scripts

To validate the correctness of my C implementations, I developed a set of small **Python verification scripts** based on **SciPy**. These scripts were designed to ensure consistency between my 2-D program and standard scientific libraries.

**Convolution checker (verify_conv2d.py)**

Uses scipy.signal.fftconvolve to compute a reference 2-D convolution in either "same" or "full" mode, and writes the result in the same assignment-compliant format as my C program (H W header, followed by values).

```
#!/usr/bin/env python3
# SciPy-based 2-D convolution checker.
# Usage:
#   python verify_conv2d_min.py -F tests_2d/f0.txt -G tests_2d/g0.txt -O ref_o0.txt
# This writes ref_o0.txt in the same "H W + values" format (3 decimals) for manual diff.

import argparse
from pathlib import Path
import numpy as np
from scipy.signal import fftconvolve

def read_hw_matrix(path: str) -> np.ndarray:
  """
  Reads matrix in assignment format:
    first line (or first two tokens): H W
    followed by H*W floats (any whitespace, any line breaks).
  Returns float64 ndarray of shape (H, W).
  """
  tokens = Path(path).read_text().split()
  if len(tokens) < 2:
    raise ValueError(f"{path}: expected 'H W' header.")
  H = int(tokens[0]); W = int(tokens[1])
  vals = list(map(float, tokens[2:]))
  if len(vals) != H * W:
    raise ValueError(f"{path}: expected {H*W} values, found {len(vals)}.")
  return np.array(vals, dtype=np.float64).reshape(H, W)

def write_hw_matrix(path: str, M: np.ndarray):
  """
  Writes matrix in assignment format:
```

```
    line 1: 'H W'
    next lines: H*W floats (3 decimals), row-major; one row per line.
  """
  H, W = M.shape
  with open(path, "w") as f:
    f.write(f"{H} {W}\n")
    for r in range(H):
      row = " ".join(f"{float(M[r, c]):.3f}" for c in range(W))
      f.write(row + ("\n" if r + 1 < H else ""))

def main():
  ap = argparse.ArgumentParser(description="Minimal 2D conv checker (SciPy fftconvolve).")
  ap.add_argument("-F", required=True, help="Path to F (input image) in H W + values format")
  ap.add_argument("-G", required=True, help="Path to G (kernel) in H W + values format")
  ap.add_argument("-O", required=True, help="Path to write output (same format)")
  args = ap.parse_args()

  F = read_hw_matrix(args.F)
  G = read_hw_matrix(args.G)

  # Match your C runs (SAME size output). Change to 'full' if needed.
  MODE = "same"
  Y = fftconvolve(F, G, mode=MODE)

  write_hw_matrix(args.O, Y)

  # Also print a quick summary so you can eyeball without opening the file.
  print(f"mode={MODE} → wrote {args.O} with shape {Y.shape} (H W)")

if __name__ == "__main__":
  main()
```

**Correlation checker (verify_corr2d_min.py)**

Uses scipy.signal.correlate to compute 2-D cross-correlation and writes the results in the same format.

```
#!/usr/bin/env python3
# SciPy-based 2-D correlation checker.
# Usage:
#   python verify_corr2d_min.py -F tests_2d/f0.txt -G tests_2d/g0.txt -O ref_corr_o0.txt
# This writes ref_corr_o0.txt in the same "H W + values" format (3 decimals) for manual diff.

import argparse
from pathlib import Path
import numpy as np
from scipy.signal import correlate

def read_hw_matrix(path: str) → np.ndarray:
  """
  Reads matrix in assignment format:
    first line (or first two tokens): H W
```

```
      followed by H*W floats (any whitespace, any line breaks).
    Returns float64 ndarray of shape (H, W).
    """
    tokens = Path(path).read_text().split()
    if len(tokens) < 2:
        raise ValueError(f"{path}: expected 'H W' header.")
    H = int(tokens[0]); W = int(tokens[1])
    vals = list(map(float, tokens[2:]))
    if len(vals) != H * W:
        raise ValueError(f"{path}: expected {H*W} values, found {len(vals)}.")
    return np.array(vals, dtype=np.float64).reshape(H, W)

def write_hw_matrix(path: str, M: np.ndarray):
    """
    Writes matrix in assignment format:
      line 1: 'H W'
      next lines: H*W floats (3 decimals), row-major; one row per line.
    """
    H, W = M.shape
    with open(path, "w") as f:
        f.write(f"{H} {W}\n")
        for r in range(H):
            row = " ".join(f"{float(M[r, c]):.3f}" for c in range(W))
            f.write(row + ("\n" if r + 1 < H else ""))

def main():
    ap = argparse.ArgumentParser(description="Minimal 2D correlation checker (SciPy correlate).")
    ap.add_argument("-F", required=True, help="Path to F (input image) in H W + values format")
    ap.add_argument("-G", required=True, help="Path to G (kernel) in H W + values format")
    ap.add_argument("-O", required=True, help="Path to write output (same format)")
    args = ap.parse_args()

    F = read_hw_matrix(args.F)
    G = read_hw_matrix(args.G)

    # Match your C runs (SAME size output). Change to 'full' if needed.
    MODE = "same"
    Y = correlate(F, G, mode=MODE)

    write_hw_matrix(args.O, Y)

    # Also print a quick summary so you can eyeball without opening the file.
    print(f"mode={MODE} → wrote {args.O} with shape {Y.shape} (H W)")

if __name__ == "__main__":
    main()
```

**Comparison tool (compare_2d.py)**

Loads two matrices (e.g., C output vs Python output), compares them element-wise, and reports summary error statistics: maximum absolute error, mean absolute error, RMSE, and number of entries outside a tolerance (atol, rtol).

```
#!/usr/bin/env python3
# Compare two 2-D matrices saved as:
#   line 1: "H W"
#   rest  : H*W floats (any whitespace), row-major.
# Prints tolerant error stats and exits 0 if within tolerance.

import argparse
from pathlib import Path
import numpy as np

def read_hw_matrix(path: str) -> np.ndarray:
    toks = Path(path).read_text().split()
    if len(toks) < 2:
        raise ValueError(f"{path}: expected 'H W' header")
    H, W = int(toks[0]), int(toks[1])
    vals = list(map(float, toks[2:]))
    if len(vals) != H * W:
        raise ValueError(f"{path}: expected {H*W} values, found {len(vals)}")
    return np.array(vals, dtype=np.float64).reshape(H, W)

def main():
    ap = argparse.ArgumentParser(description="Tolerant compare of two 2-D outputs.")
    ap.add_argument("-A", required=True, help="First matrix file (e.g., C output)")
    ap.add_argument("-B", required=True, help="Second matrix file (e.g., Python output)")
    ap.add_argument("--atol", type=float, default=1e-6, help="Absolute tolerance")
    ap.add_argument("--rtol", type=float, default=1e-6, help="Relative tolerance")
    ap.add_argument("--topk", type=int, default=5, help="Show top-k largest diffs")
    args = ap.parse_args()

    A = read_hw_matrix(args.A)
    B = read_hw_matrix(args.B)

    if A.shape != B.shape:
        print(f"SHAPE MISMATCH: {A.shape} vs {B.shape}")
        raise SystemExit(2)

    diff = A - B
    adiff = np.abs(diff)
    max_abs = float(adiff.max())
    mean_abs = float(adiff.mean())
    rmse = float(np.sqrt((diff**2).mean()))

    # tolerant “allclose” mask
    tol_mask = adiff <= (args.atol + args.rtol * np.abs(A))
    n_total = A.size
    n_bad = int((~tol_mask).sum())
```

```
    print(f"Shape: {A.shape}")
    print(f"Max |Δ|   : {max_abs:.9g}")
    print(f"Mean |Δ|  : {mean_abs:.9g}")
    print(f"RMSE      : {rmse:.9g}")
    print(f"Outside tol (atol={args.atol}, rtol={args.rtol}): {n_bad}/{n_total}")

    if n_bad:
        # report top-k offending locations
        flat_idx = np.argsort(adiff, axis=None)[::-1]
        print(f"Top {min(args.topk, n_bad)} diffs (row, col): |Δ|, A, B")
        shown = 0
        for idx in flat_idx:
            r, c = np.unravel_index(idx, A.shape)
            if adiff[r, c] > (args.atol + args.rtol * abs(A[r, c])):
                print(f"  ({r}, {c}): {adiff[r,c]:.9g}, {A[r,c]:.9g}, {B[r,c]:.9g}")
                shown += 1
                if shown >= args.topk:
                    break

    # Exit code: 0 if all within tolerance, 1 otherwise
    raise SystemExit(0 if n_bad == 0 else 1)

if __name__ == "__main__":
    main()
```

### Output:

```
(base) preet@Preets-MacBook-Pro-16 Convolution % cc -std=c11 -O2 -Wall -Wextra -Werror -o conv2d conv2d.c
(base) preet@Preets-MacBook-Pro-16 Convolution % ./conv2d -f tests_2d/f0.txt -g tests_2d/g0.txt -o output_2d/o0.txt
op=conv H=6 W=6 KH=3 KW=3 outH=6 outW=6 mode=same pad=zero cval=0 | conv_time=0.000002000 s
(base) preet@Preets-MacBook-Pro-16 Convolution % python compare_2d.py -A output_2d/o0.txt -B output_2d_py/o0.txt --atol 1e-6 --rtol 1e-6
Shape: (6, 6)
Max |Δ|   : 1.26
Mean |Δ|  : 0.407194444
RMSE      : 0.530638792
Outside tol (atol=1e-06, rtol=1e-06): 36/36
Top 5 diffs (row, col): |Δ|, A, B
  (4, 5): 1.26, 1.334, 2.594
  (3, 5): 1.19, 1.149, 2.339
  (4, 0): 1.188, 2.196, 1.008
  (5, 0): 0.998, 1.694, 0.696
  (3, 0): 0.811, 2.015, 1.204
(base) preet@Preets-MacBook-Pro-16 Convolution % python compare_2d.py -A output_2d/o0.txt -B output_2d_py_corr/o0.txt --atol 1e-6 --rtol 1e-6
Shape: (6, 6)
Max |Δ|   : 0
Mean |Δ|  : 0
RMSE      : 0
Outside tol (atol=1e-06, rtol=1e-06): 0/36
```

From this run:

- My initial **C implementation labelled as convolution** actually produced results that matched **SciPy correlation** (scipy.signal.correlate2d) rather than **SciPy convolution** (scipy.signal.fftconvolve).
- This confirmed that the loop structure in my 2-D code was computing **correlation**, not true convolution.

## Cross-Validation with LMS Samples

```
(base) preet@Preets-MacBook-Pro-16 Convolution % python compare_2d.py -A tests_2d/o0.txt -B output_2d_py_corr/o0.txt --atol 1e
-6 --rtol 1e-6
Shape: (6, 6)
Max |Δ|   : 0
Mean |Δ|  : 0
RMSE      : 0
Outside tol (atol=1e-06, rtol=1e-06): 0/36
(base) preet@Preets-MacBook-Pro-16 Convolution %
```

To confirm, I compared SciPy's correlation outputs against the **reference correlation results provided on the LMS**. These matched exactly, further reinforcing that my 2-D implementation was performing correlation.

output_2d_py > o0.txt

```
1  6 6
2  0.407 0.874 0.937 1.633 1.248 0.641
3  0.453 1.523 1.734 1.896 1.612 0.876
4  0.999 2.022 2.051 1.768 1.828 1.426
5  1.204 2.397 2.292 2.065 2.149 2.339
6  1.008 2.546 2.583 2.721 2.820 2.594
7  0.696 1.356 1.645 1.508 1.887 1.559
```

output_2d_py_corr > o0.txt

```
1  6 6
2  0.416 1.312 1.431 0.984 0.978 0.355
3  0.851 1.739 2.270 1.495 1.347 0.645
4  1.808 1.792 1.894 1.581 1.513 0.768
5  2.015 2.328 1.969 2.241 2.191 1.149
6  2.196 2.543 2.533 3.004 2.954 1.334
7  1.694 1.560 1.763 1.896 2.049 0.848
```

https://learn-ap-southeast-2-prod-fleet01-xythos.co... | scipy correlation and conv - Google Search

```
6 6
0.416 1.312 1.431 0.984 0.978 0.355
0.851 1.739 2.270 1.495 1.347 0.645
1.808 1.792 1.894 1.581 1.513 0.768
2.015 2.328 1.969 2.241 2.191 1.149
2.196 2.543 2.533 3.004 2.954 1.334
1.694 1.560 1.763 1.896 2.049 0.848
```

## Design Decision

To reconcile the mismatch between **assignment requirements (convolution)** and the **LMS-provided correlation outputs**, I introduced an explicit design choice in the 2-D program:

- A helper function flip_kernel_2d() was implemented to flip the kernel along both axes, i.e.

  G_f[u,v] = G[KH - 1 - u, \, KW - 1 - v]

  This converts correlation into true convolution when applied once before calling the correlation routine.

```
float *flip_kernel_2d(const float *G, int KH, int KW)
{
    float *Gf = (float *)malloc((size_t)KH * (size_t)KW * sizeof(float));
    if (!Gf)
    {
        fprintf(stderr, "oom flipping kernel (%dx%d)\n", KH, KW);
        exit(EXIT_FAILURE);
    }
    for (int u = 0; u < KH; u++)
        for (int v = 0; v < KW; v++)
            Gf[IDX2(u, v, KW)] = G[IDX2(KH - 1 - u, KW - 1 - v, KW)];
```

```
    return Gf;
}
```

- Two **CLI flags** were added for clarity and flexibility:
  - -conv (**default**) → flips the kernel using flip_kernel_2d(), then calls correlation.
  - -corr → bypasses the flip and computes correlation directly.

```
int parse_args(int argc, char **argv,
               const char **img_path, const char **ker_path, const char **out_path,
               long *H_req, long *W_req, long *KH_req, long *KW_req,
               unsigned long *seed, int *have_seed,
               op_kind *op, conv_mode *cmode, pad_mode *pmode, float *cval, int *have_cval)
{
    ...
}
```

This design ensures that:

1. **Correlation mode** matches SciPy's correlate2d and the LMS-provided outputs.
2. **Convolution mode** matches the mathematical definition of convolution by incorporating the flip.

> Note
>
> The kernel flip is performed **outside the timed region**. Only the correlation kernel execution (corr2d_*) is measured. This was done deliberately to keep timing results consistent between --conv and --corr runs, ensuring that performance comparisons are fair and not skewed by preprocessing overhead.

---

## OpenMP Extension

After validating the sequential 2-D implementation, I introduced **parallelism with OpenMP**. Rather than rewriting the entire program, the approach was intentionally minimal and localized:

only the **core correlation functions** (corr2d_full and corr2d_same) were modified to include #pragma omp parallel for directives. The rest of the I/O, CLI parsing, kernel flipping, and metrics logging remained identical to the sequential version.

```
/**
 * @brief 2-D FULL cross-correlation (output size: (H+KH−1)×(W+KW−1)).
 *
 * Mathematical form (no kernel flip here):
 *   Y[oi,oj] = sum_i sum_j F[i,j] * G[oi - i, oj - j], over valid overlaps only.
 *
 * Implemented as an output-driven gather with tight index bounds so that
 * (u,v) = (oi - i, oj - j) always lies in [0..KH-1]×[0..KW-1].
 * Accumulates in double, stores to float.
 */

void corr2d_full(const float *F, int H, int W,
                 const float *G, int KH, int KW,
                 float *Y)
{
    const int outH = H + KH - 1;
```

```
    const int outW = W + KW - 1;

    #pragma omp parallel for collapse(2) schedule(runtime)
    for (int oi = 0; oi < outH; oi++) {
        for (int oj = 0; oj < outW; oj++) {
            const int i0 = (oi - (KH - 1) > 0) ? (oi - (KH - 1)) : 0;
            const int i1 = (oi < (H - 1)) ? oi : (H - 1);
            const int j0 = (oj - (KW - 1) > 0) ? (oj - (KW - 1)) : 0;
            const int j1 = (oj < (W - 1)) ? oj : (W - 1);

            double acc = 0.0;
            for (int i = i0; i <= i1; i++) {
                const int u = oi - i; /* 0..KH-1 */
                #pragma omp simd reduction(+ : acc)
                for (int j = j0; j <= j1; j++) {
                    const int v = oj - j; /* 0..KW-1 */
                    acc += (double)F[IDX2(i, j, W)] * (double)G[IDX2(u, v, KW)];
                }
            }
            Y[IDX2(oi, oj, outW)] = (float)acc;
        }
    }
}

/**
 * @brief 2-D SAME cross-correlation (output size: H×W) with optional padding.
 *
 * Mathematical form (no kernel flip here):
 *   Y[i,j] = sum_{u=0..KH-1} sum_{v=0..KW-1} F[i + (u - cH), j + (v - cW)] * G[u,v]
 * where cH = floor(KH/2), cW = floor(KW/2).
 *
 * Implementation trims (u,v) ranges so (ii,jj) = (i + u - cH, j + v - cW) stays in bounds.
 * For PAD_CONST, adds cval times the sum of G over the out-of-bounds area.
 * Accumulates in double, stores to float.
 */
void corr2d_same(const float *F, int H, int W,
                 const float *G, int KH, int KW,
                 float *Y,
                 pad_mode pmod, float cval)
{
    const int cH = KH / 2, cW = KW / 2;

    #pragma omp parallel for collapse(2) schedule(runtime)
    for (int i = 0; i < H; i++) {
        for (int j = 0; j < W; j++) {
            int u0 = cH - i; if (u0 < 0) u0 = 0;
            int u1 = (H - 1 + cH) - i; if (u1 > KH - 1) u1 = KH - 1;
            int v0 = cW - j; if (v0 < 0) v0 = 0;
            int v1 = (W - 1 + cW) - j; if (v1 > KW - 1) v1 = KW - 1;

            double acc = 0.0;
```

```
        for (int u = u0; u <= u1; u++) {
            const int ii = i + (u - cH);
            #pragma omp simd reduction(+ : acc)
            for (int v = v0; v <= v1; v++) {
                const int jj = j + (v - cW);
                acc += (double)F[IDX2(ii, jj, W)] * (double)G[IDX2(u, v, KW)];
            }
        }

        if (pmod == PAD_CONST) {
            double pad_sum = 0.0;
            for (int u = 0; u < u0; u++)
                for (int v = 0; v < KW; v++)
                    pad_sum += (double)G[IDX2(u, v, KW)];
            for (int u = u1 + 1; u < KH; u++)
                for (int v = 0; v < KW; v++)
                    pad_sum += (double)G[IDX2(u, v, KW)];
            for (int u = u0; u <= u1; u++) {
                for (int v = 0; v < v0; v++)
                    pad_sum += (double)G[IDX2(u, v, KW)];
                for (int v = v1 + 1; v < KW; v++)
                    pad_sum += (double)G[IDX2(u, v, KW)];
            }
            acc += (double)cval * pad_sum;
        }

        Y[IDX2(i, j, W)] = (float)acc;
      }
    }
}
```

This demonstrates the *light-touch modification philosophy*: I didn't alter the indexing logic or the mathematical correctness of the kernel, only parallelized the independent output loops.

The **SAME mode** was handled similarly, parallelizing over (i,j) output coordinates.

---

## Verification

To ensure correctness of the OpenMP implementation, I generated **random but reproducible inputs** by fixing the RNG seed (-s 42). This guarantees both the sequential and parallel runs receive identical F and G matrices.

```
parallels@ubuntu-linux-2404:~/Documents/GitHub/Convolution$ cc -std=c11 -Wall -Wextra -fopenmp -o conv2d_omp conv2d_omp.c
parallels@ubuntu-linux-2404:~/Documents/GitHub/Convolution$ export OMP_NUM_THREADS=8
parallels@ubuntu-linux-2404:~/Documents/GitHub/Convolution$ export OMP_SCHEDULE=static
parallels@ubuntu-linux-2404:~/Documents/GitHub/Convolution$ ./conv2d_omp -H 256 -W 256 -kH 5 -kW 5 -o out_omp.txt -s 42
op=conv H=256 W=256 KH=5 KW=5 outH=256 outW=256 mode=same pad=zero cval=0 | conv_time=0.004511333 s
parallels@ubuntu-linux-2404:~/Documents/GitHub/Convolution$ cc -std=c11 -Wall -Wextra -o conv2d conv2d.c
parallels@ubuntu-linux-2404:~/Documents/GitHub/Convolution$ ./conv2d -H 256 -W 256 -kH 5 -kW 5 -o out_seq.txt -s 42
op=conv H=256 W=256 KH=5 KW=5 outH=256 outW=256 mode=same pad=zero cval=0 | conv_time=0.006938500 s
```

```
Shape: 256 x 256
Max |Δ|   : 0
Mean |Δ|  : 0
RMSE      : 0
Outside tol (atol=1e-06, rtol=1e-06): 0/65536
PASS: arrays match within tolerance
parallels@ubuntu-linux-2404:~/Documents/GitHub/Convolution$
```

The element-wise comparison confirmed that **sequential and OMP outputs matched exactly** within floating-point tolerance, validating the correctness of the parallel implementation.

### Porting enhancements from 1-D program

Further enhancements from a sequential 2-D convolution/correlation program to an OpenMP-enabled implementation was guided by the design principles already established in the 1-D case.

## Flexible Runtime Scheduling

To ensure portability and user control, the scheduling policy is not hard-coded. Instead, schedule(runtime) is employed, which delegates the choice of scheduling strategy to the user or environment. The program supports CLI arguments to explicitly configure threads (--threads), scheduling kind (--schedule), and chunk size (--chunk).

A “smart default” chunk size is calculated when unspecified:

chunk = [outH X outW]/ [4 x threads]

This ensures a balanced workload while avoiding excessive scheduling overhead.

## NUMA and Cache-Aware Considerations

The program incorporates **first-touch initialization** of the output matrix before kernel execution:

```
#pragma omp parallel for schedule(static)
for (int i = 0; i < outH * outW; i++) {
   Y[i] = 0.0f;
}
```

This guarantees that memory pages are allocated to the threads that will later compute on them, reducing cross-socket memory traffic in NUMA systems.

In addition, the choice of iteration order (i,j) ensures that threads access data contiguously in row-major format, improving spatial locality and minimizing cache misses. Since adjacent output elements reuse overlapping regions of the input matrix and share the same kernel, cache reuse is naturally exploited.

## FLOPs-Based Performance Metrics

To provide more meaningful performance insights, the program reports floating-point throughput in GFLOP/s in addition to execution time. The total floating-point operations are approximated as:

FLOPs = 2 x outH x outW x KH x KW

The GFLOP/s rate is then computed and reported alongside timing results. This normalization allows for fairer comparisons across problem sizes and hardware configurations.

Through these enhancements, the 2-D OpenMP implementation benefits from:

- Independent parallelism over output elements.
- SIMD acceleration of innermost loops.
- Flexible runtime scheduling with user-tunable defaults.
- NUMA-friendly first-touch initialization and improved cache locality.
- FLOPs-based performance reporting for normalized benchmarking.
- Rigorous fairness in timing methodology.

These optimizations collectively transform the baseline sequential 2-D program into a scalable, cache-aware, and performance-transparent OpenMP implementation, while preserving correctness and reproducibility.

### Verification of complete enhance program

```
parallels@ubuntu-linux-2404:~/Documents/GitHub/Convolution$ cc -std=c11 -Wall -Wextra -fopenmp -o conv2d_omp conv2d_omp.c
parallels@ubuntu-linux-2404:~/Documents/GitHub/Convolution$ ./conv2d_omp -H 256 -W 256 -kH 5 -kW 5 -o out_omp.txt -s 42
op=conv H=256 W=256 KH=5 KW=5 outH=256 outW=256 mode=same pad=zero cval=0 | conv_time=0.003771083 s | 0.869 GFLOP/s | OMP{threads=1 sched=st
atic chunk=16384}
parallels@ubuntu-linux-2404:~/Documents/GitHub/Convolution$ python3 - <<'PY'
import sys, math

A = "out_seq.txt"
B = "out_omp.txt"
ATOL = 1e-6
RTOL = 1e-6
TOPK = 5

def read_hw(path):
    with open(path, "r") as f:
        toks = f.read().split()
    if len(toks) < 2:
        raise ValueError(f"{path}: expected 'H W' header")
    H, W = int(toks[0]), int(toks[1])
    vals = list(map(float, toks[2:]))
    if len(vals) != H * W:
        raise ValueError(f"{path}: expected {H*W} values, found {len(vals)}")
    return H, W, vals

def stats(a, b, H, W):
    assert len(a) == len(b) == H*W
    diffs = [ai - bi for ai, bi in zip(a, b)]
    adiffs = [abs(d) for d in diffs]
    mx = max(adiffs) if adiffs else 0.0
    mean_abs = sum(adiffs) / (H*W) if adiffs else 0.0
    rmse = math.sqrt(sum(d*d for d in diffs) / (H*W)) if diffs else 0.0
PY  sys.exit(0): arrays match within tolerance"), {bi:.9g}")|Δ|, A, B"))d to A
Shape: 256 x 256
Max |Δ|   : 0
Mean |Δ|  : 0
RMSE      : 0
Outside tol (atol=1e-06, rtol=1e-06): 0/65536
PASS: arrays match within tolerance
parallels@ubuntu-linux-2404:~/Documents/GitHub/Convolution$
```

The performance improvements are evident in the reduction of execution time from conv_t = 0.00451s to conv_t = 0.00377s after implementing these enhancements.

## For detailed analysis and experimentation on Kaya please refer to Analysis.ipynb file or Analysis.html